Introduction to inverse reinforcement learning

Eiji Uchibe

Dept. of Brain Robot Interface

ATR Computational Neuroscience Labs.

Together to the Top for Future Society



Learning objectives

By the end of this lecture, you will be able to:

1. Explain what Inverse Reinforcement Learning (IRL) estimates from behavior (reward / preference / goal).
2. Describe the Markov Decision Process (MDP) components (state, action, transition, reward, policy) in plain terms.
3. Explain why IRL is ill-posed (reward ambiguity) and how MaxEnt resolves it.
4. Interpret an estimated reward as “what the agent cares about” in real applications (healthcare, animals, sports).

Prerequisites: basic probability and optimization

What is Reinforcement Learning (RL)?

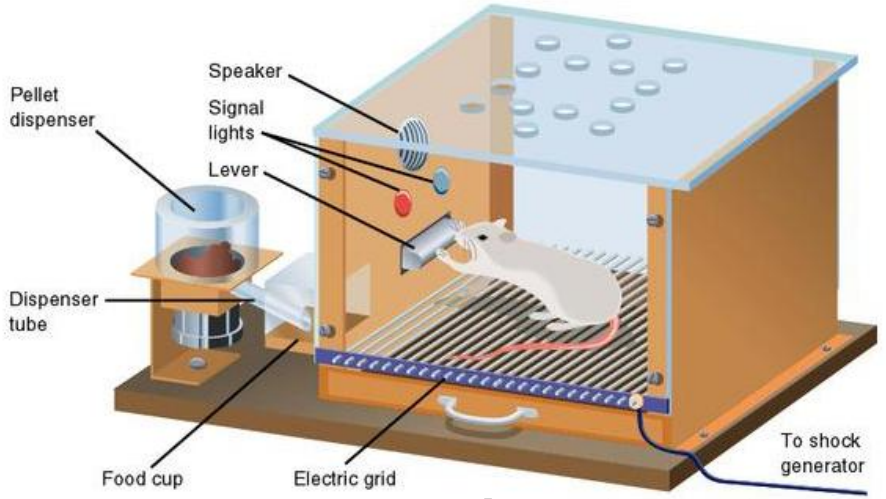
RL is an algorithmic framework for finding an optimal policy (controller) by trial and error

→ A learning framework for artificial agents

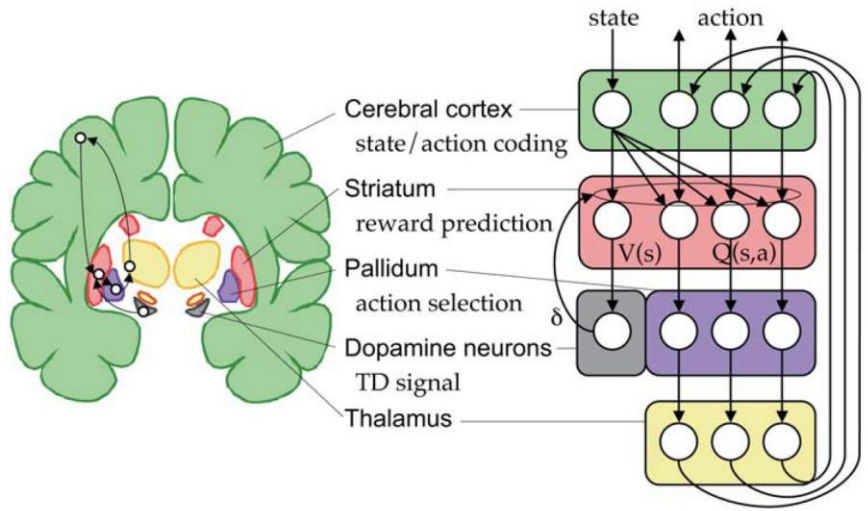
RL is a computational model of goal-directed decision-making in humans and animals under uncertainty

Inspired by psychology

- Thorndike's law of effect
- Skinner's principle of reinforcement



[Simply Psychology]

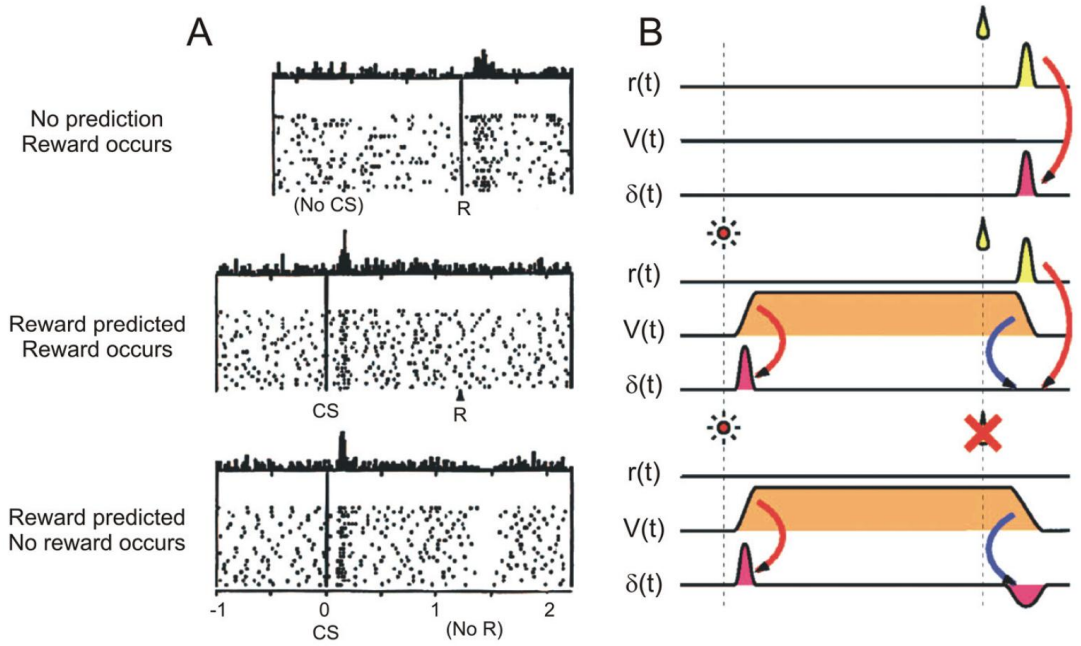


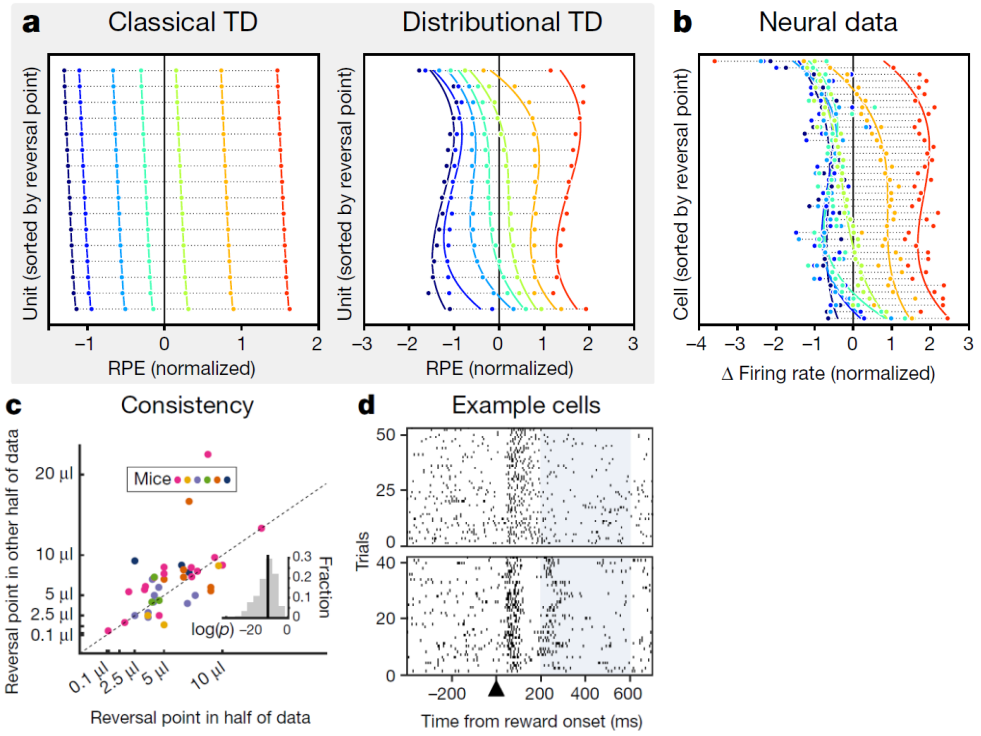
K. Doya (2007). Reinforcement learning: Computational theory and biological mechanisms. HFSP Journal, vol. 1, no. 1, pp. 30–40.

Reinforcement learning in neuroscience

Dopamine neurons encode reward prediction errors (RPEs)

Recent work suggests a distributional representation of value/RPE in the brain



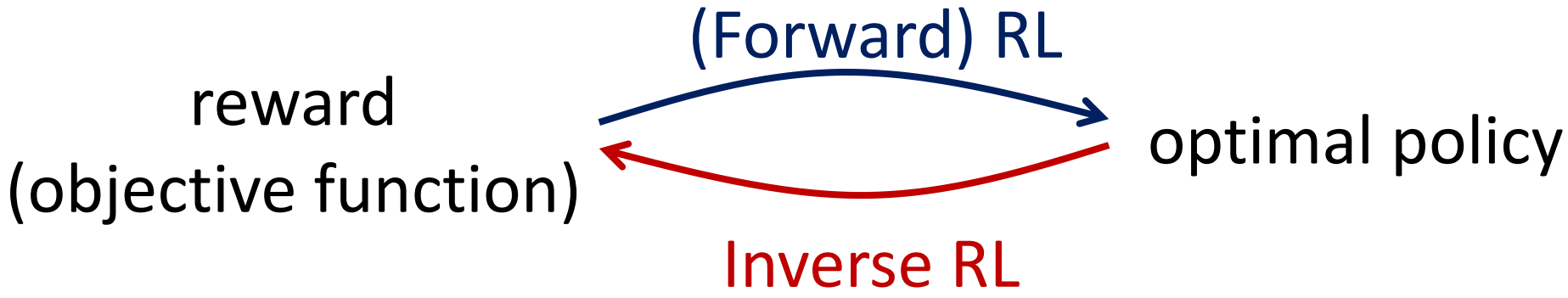


Schultz, W.P., Dayan, P., and Montague, P.R. (1997). A Neural Substrate of Prediction and Reward. *Science* 275, no. 5306: 1593–99.

Dabney, W., Kurth-Nelson, Z., Uchida, N. et al. (2020). A distributional code for value in dopamine-based reinforcement learning. *Nature*, 577, 671–675.

Inverse Reinforcement Learning (Inverse RL)

Estimate a reward function from observed behaviors generated by an optimal policy



We use inverse RL when we can observe behavior but cannot directly measure the objective (preference, goal, cost)

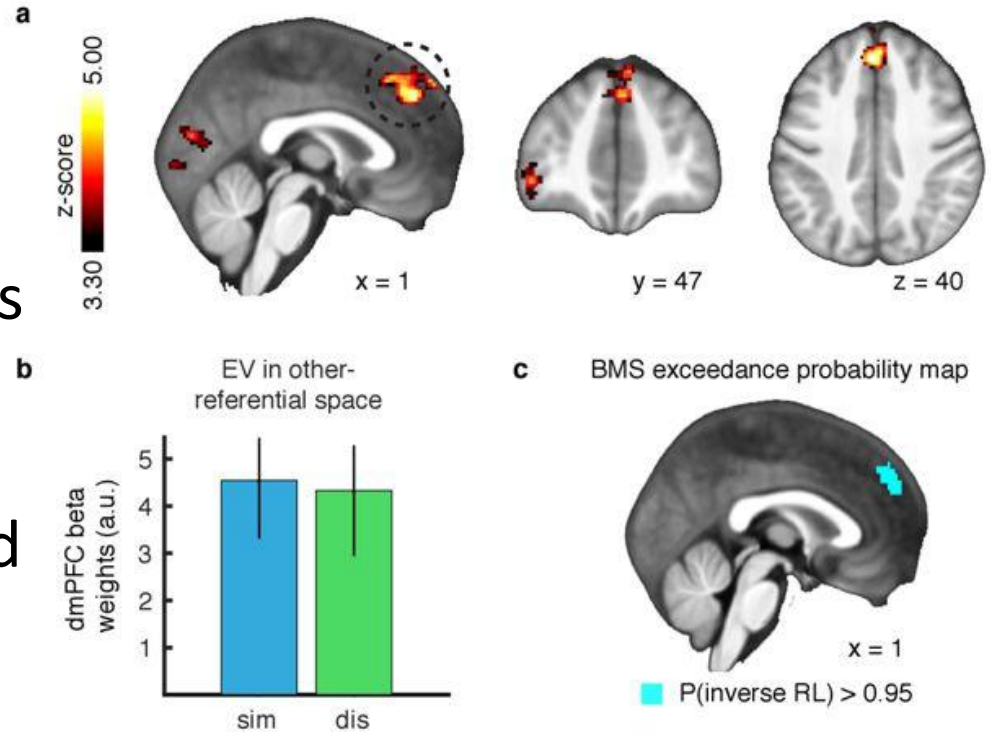
Inverse RL is Ill-posed: **the reward is not uniquely identifiable from behavior alone**

Application: Inverse RL implemented in the human brain

Background: Humans learn by watching others, but it was unclear if the brain simply copies actions (imitation) or mathematically calculates the hidden goals driving those actions (inverse RL)

Objective: This study used fMRI to determine if people use inverse RL to learn from agents with different preferences and to identify brain regions

Contribution: They suggested that humans use inverse RL to infer hidden rewards and discovered dmPFC processes values from another person's perspective rather than the observer's own



Collette et al. (2017). Neural computations underlying inverse reinforcement learning in the human brain. eLife.

Glossary (1/2)

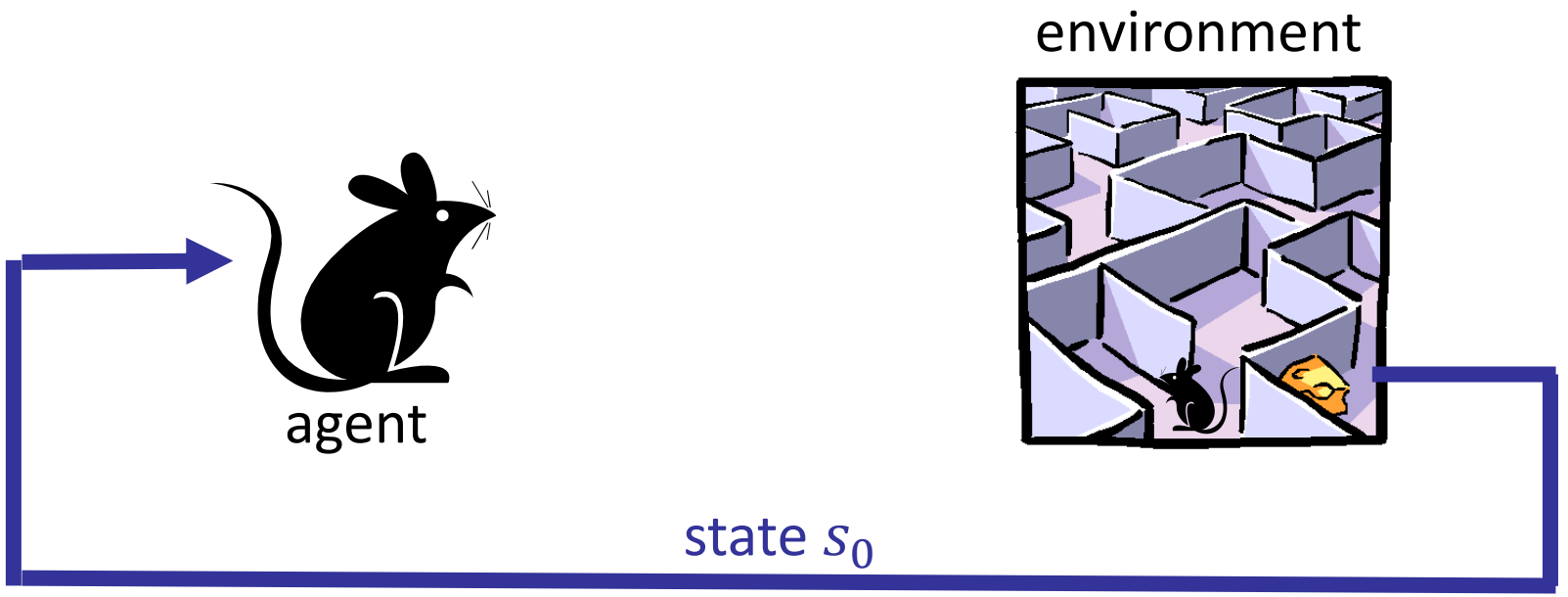
Term	Explanation	Medical & robotics examples
Initial state probability (p_0)	Starting condition	The patient's condition at the start of a shift
State transition probability (p_T)	World dynamics	The physics of the environment. E.g., how room temperature drops when a window is opened
Trajectory (τ)	Experience history	The complete story or sequence of events, recording every state visited and every action taken from start to finish
Return ($\sum_t r_t$)	Total success score	The sum of all rewards collected over the entire trajectory; the final grade for the agent's performance

Glossary (2/2)

Term	Explanation	Medical & robotics examples
State (s)	Current situation	A patient's pain level, fatigue, heart rate, or the current room temperature
Action (a)	Intervention/Movement	Clinical interventions like adjusting medication, opening a window, or a robot changing its rout
Reward ($r(s, a)$)	Quality score (success)	Measures of how good the result was, such as pain relief, avoiding collision, or saving energy
Policy ($\pi(\ a \mid s\ )$)	Behavior strategy	The logical rule of habit used to decide which action to take based on the current situation

Modeling agent-environment interaction by a Markov Decision Process (MDP)

At time $t = 0$, an initial state of an environment is determined by an initial state distribution: $s_0 \sim p_0(\cdot)$



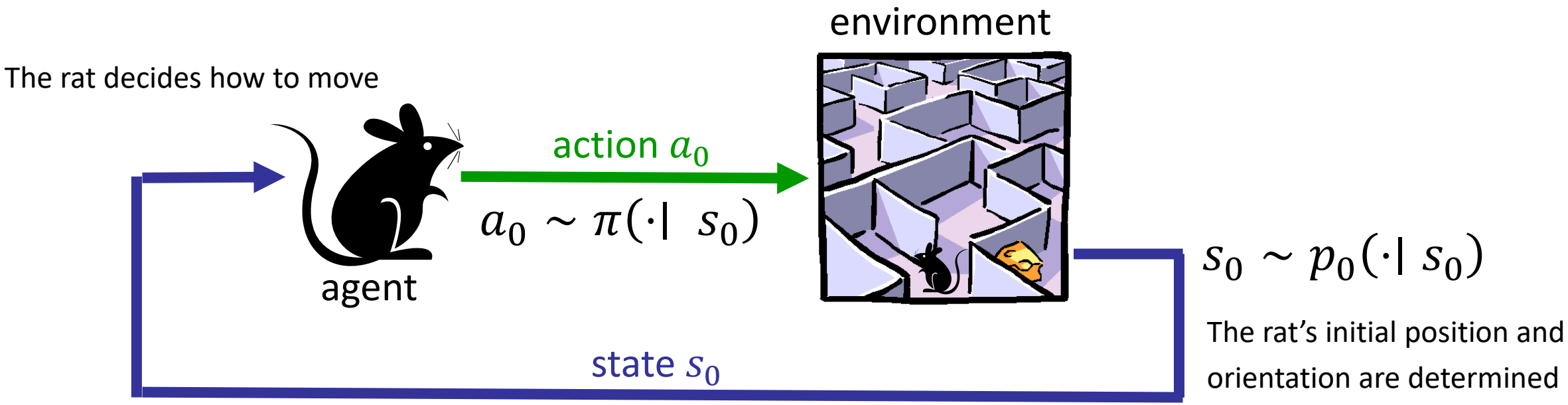
$s_0 \sim p_0(\cdot | s_0)$

The rat's initial position and orientation are determined

trajectory $\tau = \{s_0\}$

Modeling agent-environment interaction by a MDP

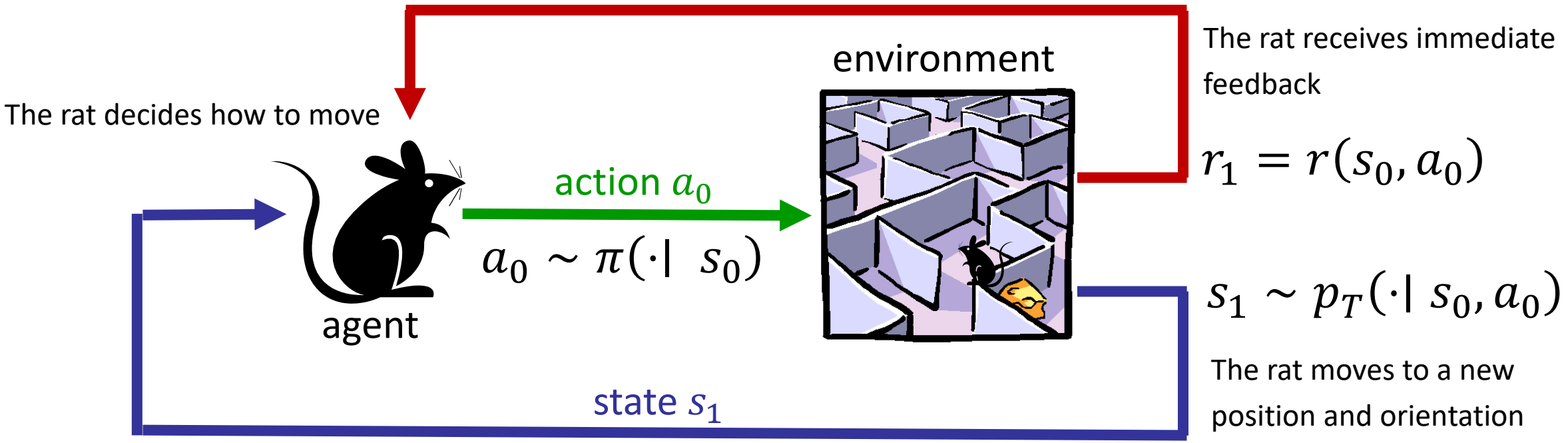
At time $t = 0$, the rat makes a decision by policy, an action is determined by a stochastic policy: $a_0 \sim \pi(\cdot|\ s_0)$



trajectory $\tau = \{s_0, a_0\}$

Modeling agent-environment interaction by a MDP

As time goes from $t = 0$ to $t = 1$, the environment changes in response to the action taken by the agent and provides feedback reward based on the outcome: $s_1 \sim p_T(\cdot|\, s_0, a_0), r_1 = r(s_0, a_0)$

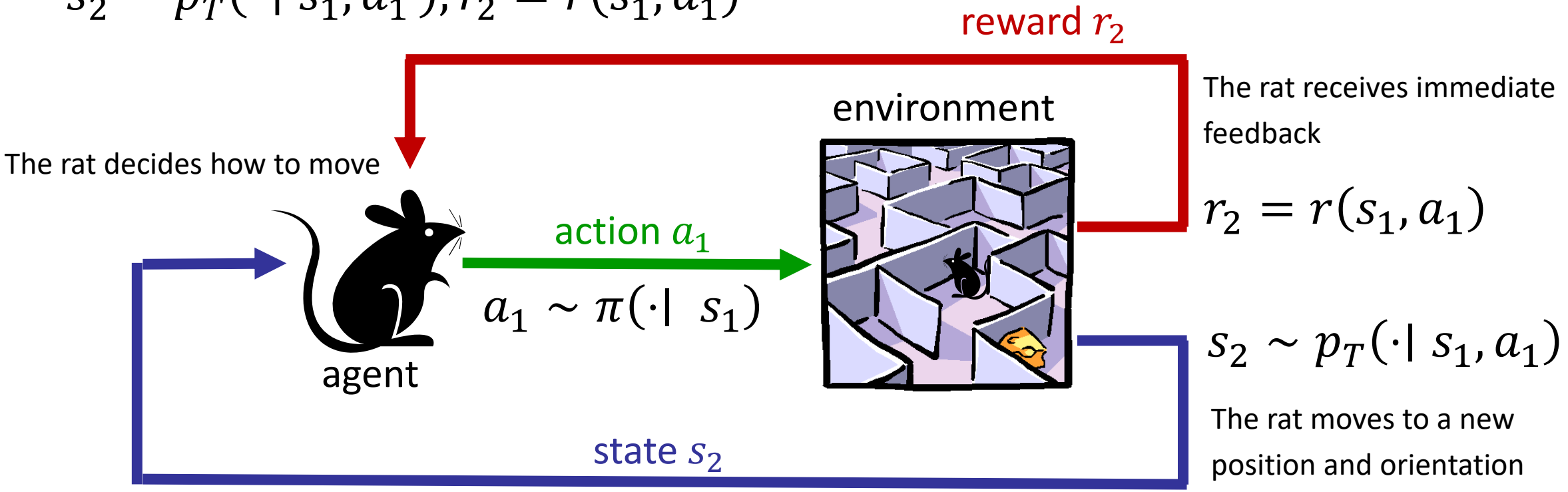


trajectory $\tau = \{s_0, a_0, r_1, s_1\}$

Modeling agent-environment interaction by a MDP

At time $t = 1$, the rat decides an action and obtains a new experience according to the environmental change: $a_1 \sim \pi(\cdot|\, s_1)$,

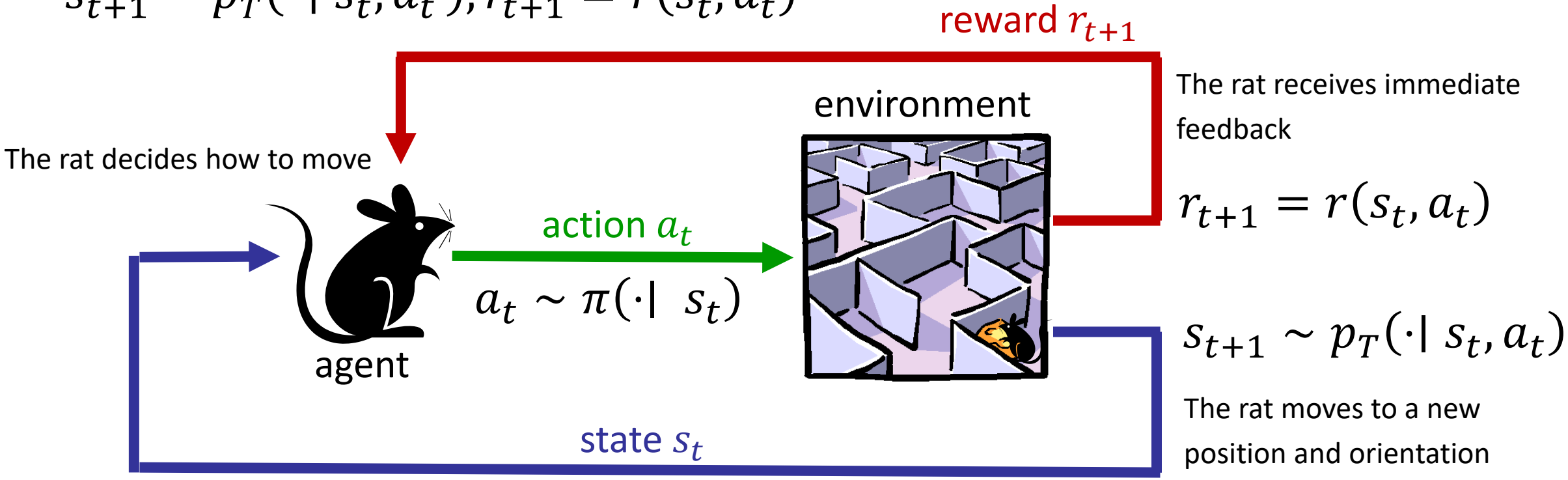
$s_2 \sim p_T(\cdot|\, s_1, a_1), r_2 = r(s_1, a_1)$



trajectory $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, s_2\}$

Modeling agent-environment interaction by a MDP

By iterating the interaction until $t = T - 1$, the rat obtains a trajectory, which is state-action-reward sequences: $a_t \sim \pi(\cdot|\, s_t)$, $s_{t+1} \sim p_T(\cdot|\, s_t, a_t), r_{t+1} = r(s_t, a_t)$



trajectory $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots, s_{T-1}, a_{T-1}, r_T, s_T\}$

The objective of RL

Find an optimal policy π^* that maximizes the expected (optionally discounted) return along the trajectories:

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_{p(\tau)} \left[\sum_{t=1}^{T} r_t \right]$$

where τ denotes a trajectory of states, actions, and rewards:

$$\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, s_2, \ldots, s_{T-1}, a_{T-1}, r_T, s_T\}$$

and $p(\tau)$ is the probability of observing a state-action-reward sequence

The rewards specify how the agent make decisions

Goal of inverse RL

Infer a reward function that aligns with the observed state-action sequences (trajectories/demonstrations)

Maximum entropy inverse RL (MaxEnt IRL) assumes that the probability of observing trajectories is represented by

$$p(\tau \mid \boldsymbol{w}) = \frac{1}{Z(\boldsymbol{w})} p_0(s_0) \prod_{t=0}^{T-1} p_T(s_{t+1} \mid s_t, a_t) \exp\left(\sum_{t=1}^{T} r(s_t, a_t; \boldsymbol{w})\right)$$

where $\boldsymbol{w}$ is the parameter vector of the reward function and $Z(\boldsymbol{w})$ is the partition function

$\boldsymbol{w}$ is trained via maximum likelihood estimation from expert trajectories

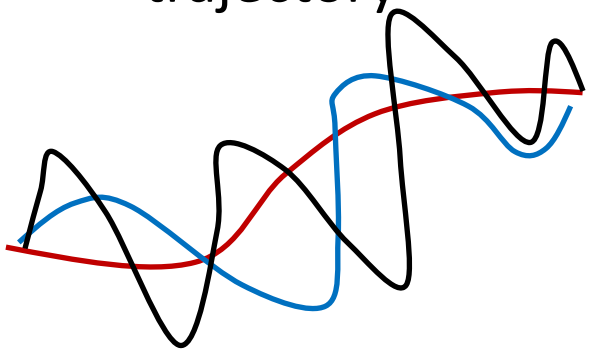
Ziebart, et al. (2008). Maximum entropy inverse reinforcement learning. In Proc. of AAAI, pp. 1433-1438.

Interpretation of $p(\tau \mid w)$

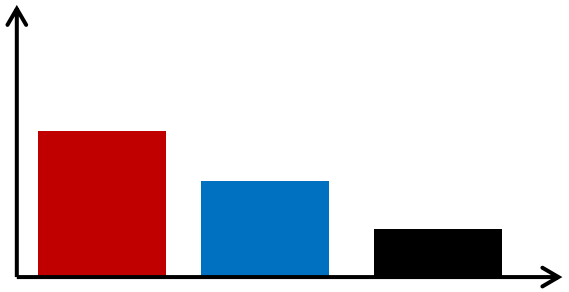
Trajectories with higher returns are exponentially more preferred

Trajectories with equivalent returns have equal probabilities

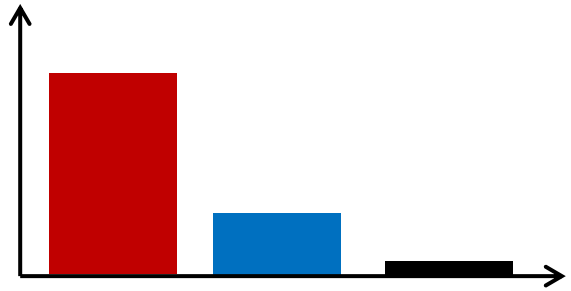
trajectory



$R(\tau_i)$ total rewards



$\Pr(\tau_i)$ probability



MLE formulation and its difficulty

MLE objective is given by

$$\max_{\boldsymbol{w}} \mathbb{E}_{\tau \setminus r \sim \pi_E}[\ln p(\tau \mid \boldsymbol{w})] = \mathbb{E}_{\tau \setminus r \sim \pi_E}\left[\sum_{t=1}^{T} r(s_t, a_t; \boldsymbol{w})\right] - \ln Z(\boldsymbol{w})$$

where $\mathbb{E}_{\tau \setminus r \sim \pi_E}$ is the expectation of state-action sequences generated by the expert policy π_E

The gradient of log-likelihood is represented by

$$\mathbb{E}_{\tau \sim \pi_E}\left[\sum_{t=1}^{T} \frac{\partial r(s_t, a_t; \boldsymbol{w})}{\partial \boldsymbol{w}}\right] - \frac{\partial \ln Z(\boldsymbol{w})}{\partial \boldsymbol{w}}$$

How do we compute $\partial \ln Z(\boldsymbol{w}) / \partial \boldsymbol{w}$?

Evaluation of $\partial \ln Z(\boldsymbol{w})/\partial \boldsymbol{w}$

Under the assumption of $p(\tau \mid \boldsymbol{w})$, we obtain

$$\frac{\partial \ln Z(\boldsymbol{w})}{\partial \boldsymbol{w}} = \frac{1}{Z(\boldsymbol{w})}\frac{\partial Z(\boldsymbol{w})}{\partial \boldsymbol{w}}$$

$$= \frac{1}{Z(\boldsymbol{w})}\sum_{\tau} p_0(s_0)\prod_{t=0}^{T-1} p_T(s_{t+1} \mid s_t, a_t)\exp\left(\sum_{t=1}^{T} r(s_t, a_t; \boldsymbol{w})\right)\sum_{t=1}^{T}\frac{\partial r(s_t, a_t; \boldsymbol{w})}{\partial \boldsymbol{w}}$$

$$= \sum_{\tau} p(\tau \mid \boldsymbol{w})\sum_{t=1}^{T}\frac{\partial r(s_t, a_t; \boldsymbol{w})}{\partial \boldsymbol{w}} = \mathbb{E}_{\tau\sim p(\tau|w)}\left[\sum_{t=1}^{T}\frac{\partial r(s_t, a_t; \boldsymbol{w})}{\partial \boldsymbol{w}}\right]$$

Consequently, $\partial \ln Z(\boldsymbol{w})/\partial \boldsymbol{w}$ is expressed as the expectation along the trajectories generated by the policy trained by $r(s, a; \boldsymbol{w})$

MaxEnt Inverse RL algorithm

Finally, w is updated by gradient ascent, where the gradient is given by

$$\underbrace{\mathbb{E}_{\tau\sim\pi_E}\left[\sum_{t=1}^{T}\frac{\partial r(s_t,a_t;w)}{\partial w}\right]}_{\text{Expectation of the gradient of the sum of rewards along the expert trajectories}} - \underbrace{\mathbb{E}_{\tau\sim p(\tau|w)}\left[\sum_{t=1}^{T}\frac{\partial r(s_t,a_t;w)}{\partial w}\right]}_{\text{Expectation of the gradient of the sum of rewards along the trajectories generated by the policy trained by } r(s,a;w)}$$

To evaluate the second term, we need the trajectories generated by the optimal policy under the current reward estimate!

Therefore, the RL problem should be solved for every update of w, **making inverse RL time-consuming**

Representation of the reward function

The reward function is often modeled by the linear function approximator:

$$r(s, a; \boldsymbol{w}) \triangleq \sum_{i=1}^{N} w_i \phi_i(s, a) = \boldsymbol{w}^\top \boldsymbol{\phi}(s, a)$$

where ϕ is called the reward feature vector

We can encode domain knowledge by designing $\boldsymbol{\phi}$

Of course, we can use nonlinear function approximators such as deep neural networks and Gaussian Processes

Simulation: FrozenLake task



Discrete-state and discrete-action MDP task

provided by Gymnasium

- Simplify the task by considering a deterministic MDP

Positive reward (+1) for entering the goal

from left

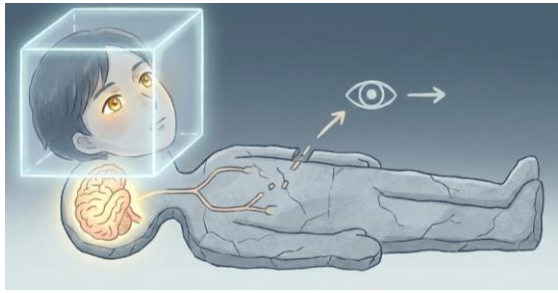
0 otherwise

An episode ends when entering a hole

- 0 reward is given to the agent

If you have a Google account, go to https://uchibe.github.io/BILT-B/

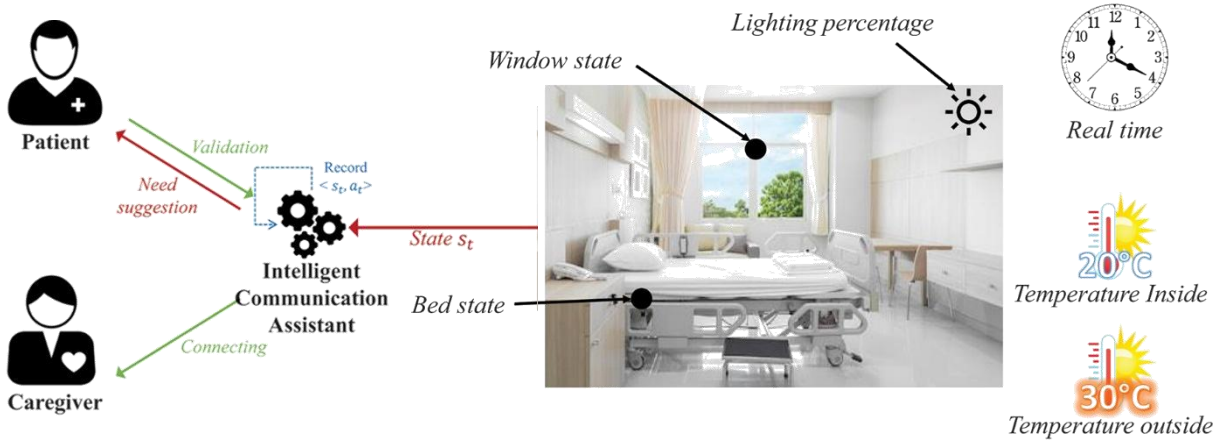
Application: Smart health-care assistants



Background: People with Locked-in Syndrome (LIS) are paralyzed and cannot speak, making it very slow and tiring to communicate their needs using only eye movements

Objective: Create a smart AI assistant that uses ambient sensors to infer latent needs (e.g., temperature) and automatically predict what the patient needs. Estimated rewards provide an **interpretable preference model** useful for clinical decision support

Contribution: The proposed system successfully predicts patient needs with over 90% accuracy



Hantous et al. (2022). Detecting Physiological Needs Using Deep Inverse Reinforcement Learning. *Applied Artificial Intelligence*.

Features, actions, and reward network

feature vector
$\boldsymbol{\phi}(s) \in \mathbb{R}^6$

Environment's feature
Time
Room Temperature
Exterior Temperature
Lighting percentage
Window state
Bed state

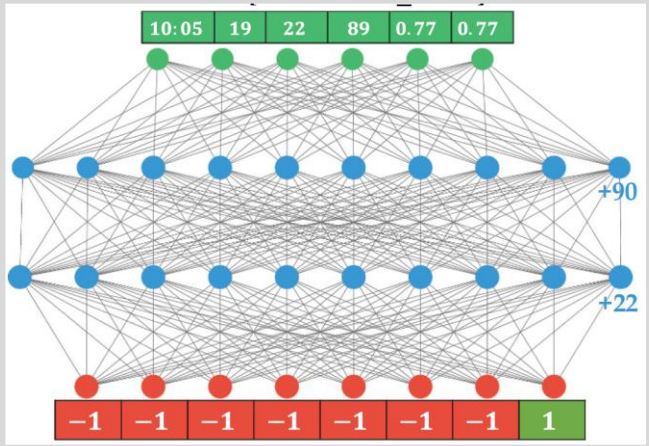
eight discrete actions

ID	Action: patient's need
0	Open the window
1	Close the window
2	Change the bed position
3	Move
4	Drink
5	Set the temperature
6	Sleep
7	Adjust the light

State s is not observed directly, but six corresponding features are available

Eight discrete actions are available

six features of state s



$r_a(s)$: reward of action a at state s

The reward function is approximated by the neural network shown in the left panel

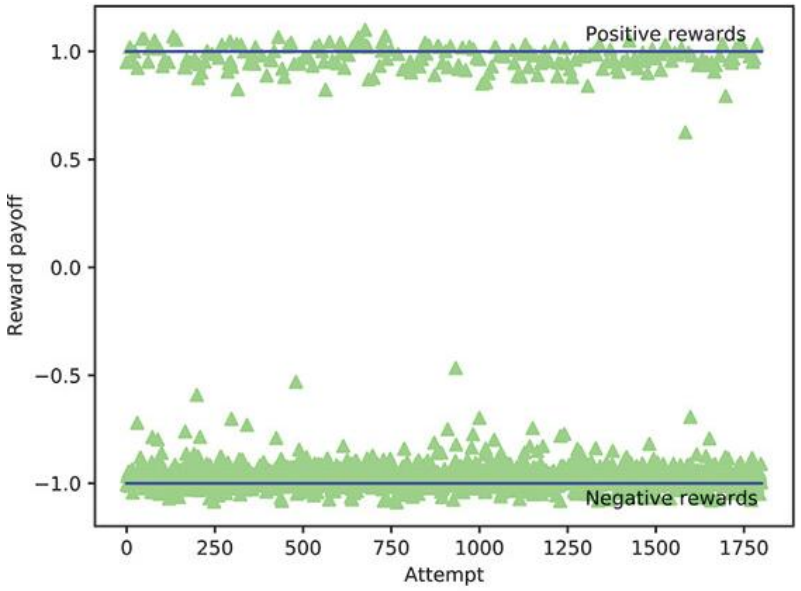
The input is six features of state s, and the output is a list of $r_a(s)$

Smart health-care assistants

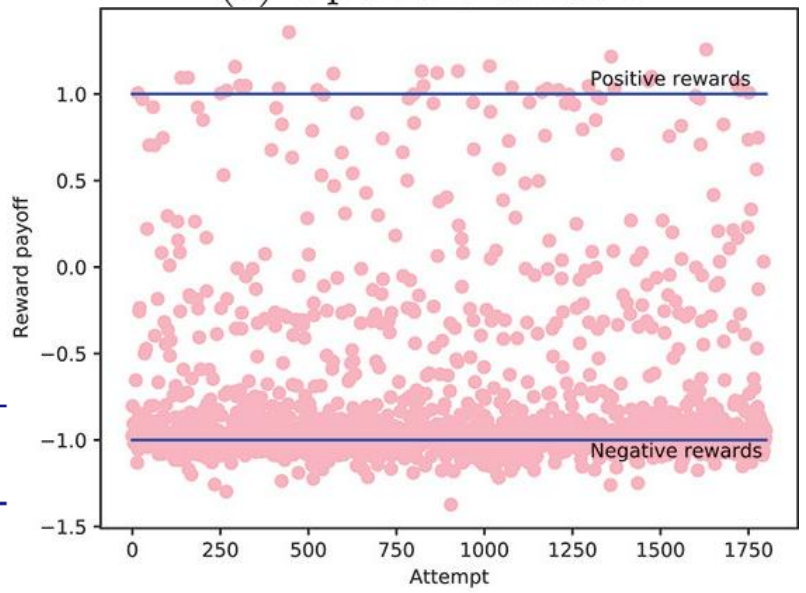
The right figures represent $\left\|\boldsymbol{R}_E - \widehat{\boldsymbol{R}}_e\right\|$, where $\boldsymbol{R}_E$ and $\widehat{\boldsymbol{R}}_e$ denote the true and estimated reward vector, respectively

- The need to “open the window” was well predicted
- The proposed method failed to predict the need of “sleep”

Table 6 shows the summary of the results



(a) Open the window



(b) Sleep

Table 6. Percentage of giving the highest reward to the fitable action.

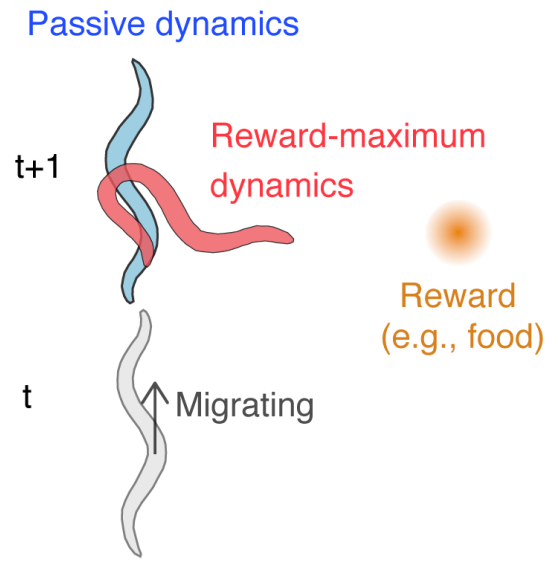
Action (need)	*Open the window*	*Close the window*	*Change the bed*	*Move*	*Drink*	*Set the temperature*	*Sleep*	*Adjust the light*
Total Number	200	200	200	200	200	400	200	200
Matching	100%	100%	98%	96.5%	88.5%	89.5%	56%	96%

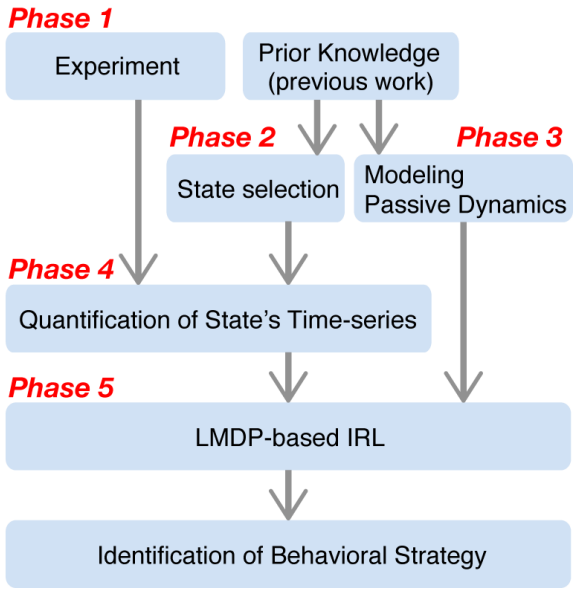
Investigation of *C. elegans* thermotactic behavior

Background: It is difficult to understand the internal rules or hidden rewards that guide animal decisions just by observing their free movements in natural environments

Objective: The goal was to use inverse RL to decode the strategy of C. elegans by analyzing how they move in response to temperature changes

Contribution: They discovered that fed worms calculate temperature changes to navigate efficiently, whereas starved worms only react to absolute temperature to escape





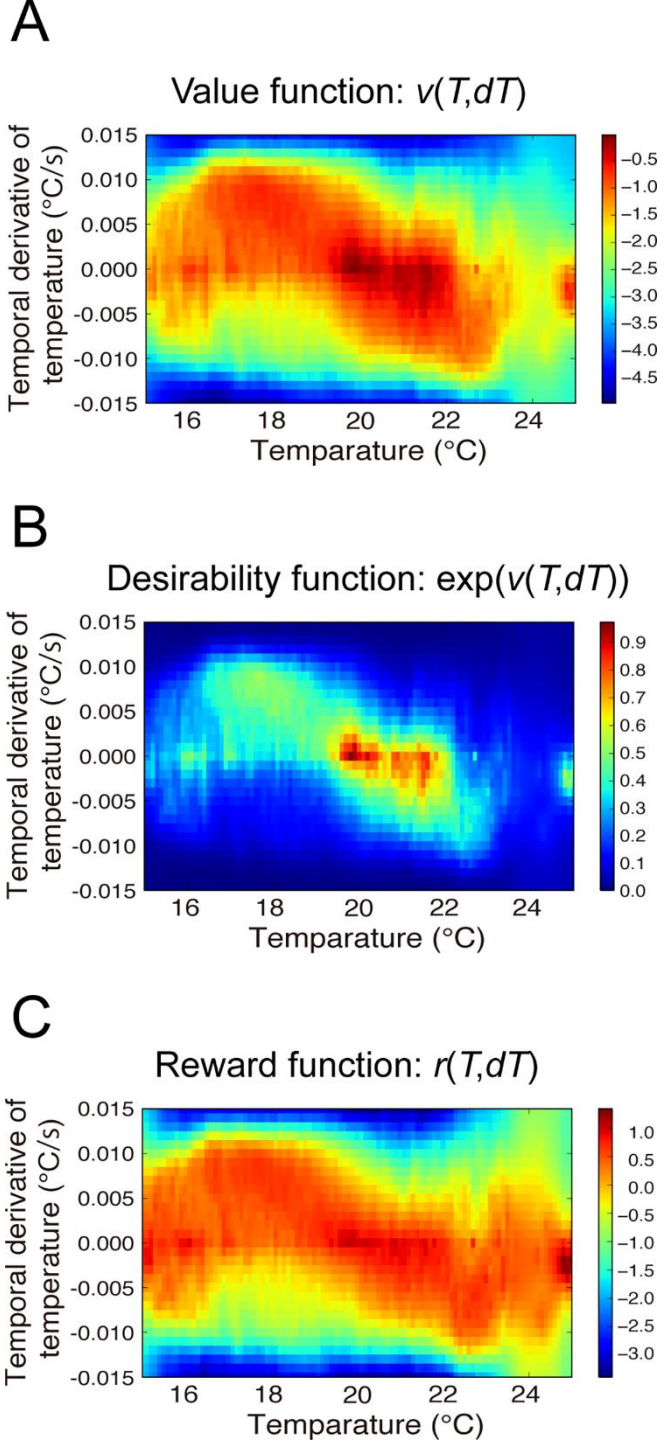
Yamaguchi et al. (2018). Identification of animal behavioral strategies by inverse reinforcement learning. PLoS Computational Biology. [Press release written in Japanese]

Behavioral strategy of fed wild-type worms

Two basic strategies are found:

Directed Migration (DM): Worms efficiently reached specific temperatures, which explains their thermotactic behavior when fed

Isothermal Migration (IM). Worms moved along a constant temperature, reflecting isothermal tracking, as well-observed in previous studies

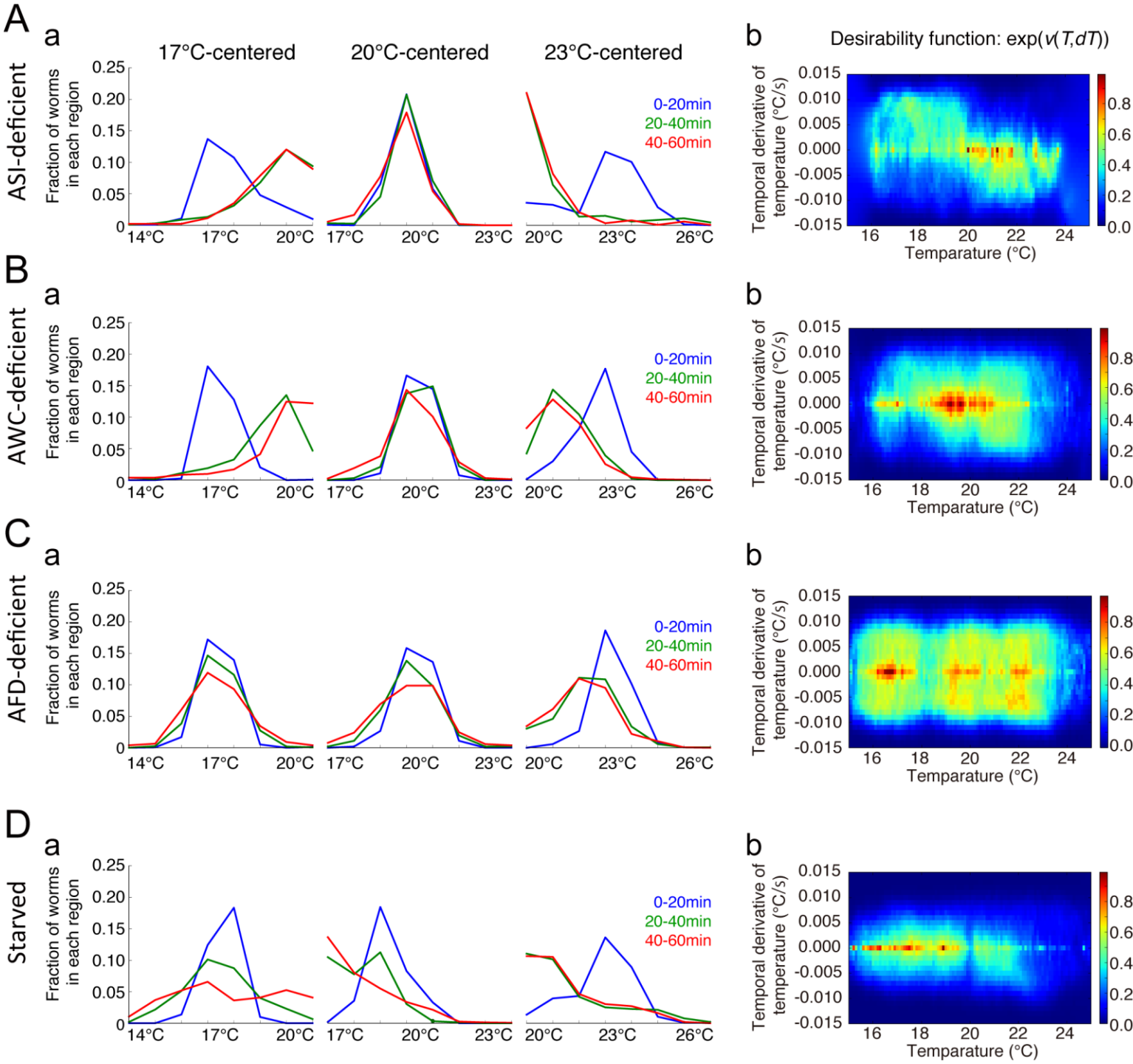


Yamaguchi et al. (2018). Identification of animal behavioral strategies by inverse reinforcement learning. PLoS Computational Biology. [Press release written in Japanese]

Strategies of mutant and starved worms

AFD-deficient worms lose the DM strategy (lacking the diagonal structure) and become cryophilic (preferring cold)

Starved worms avoid the cultivation temperature using only absolute temperature cues without relying on the temperature derivative used by fed worms



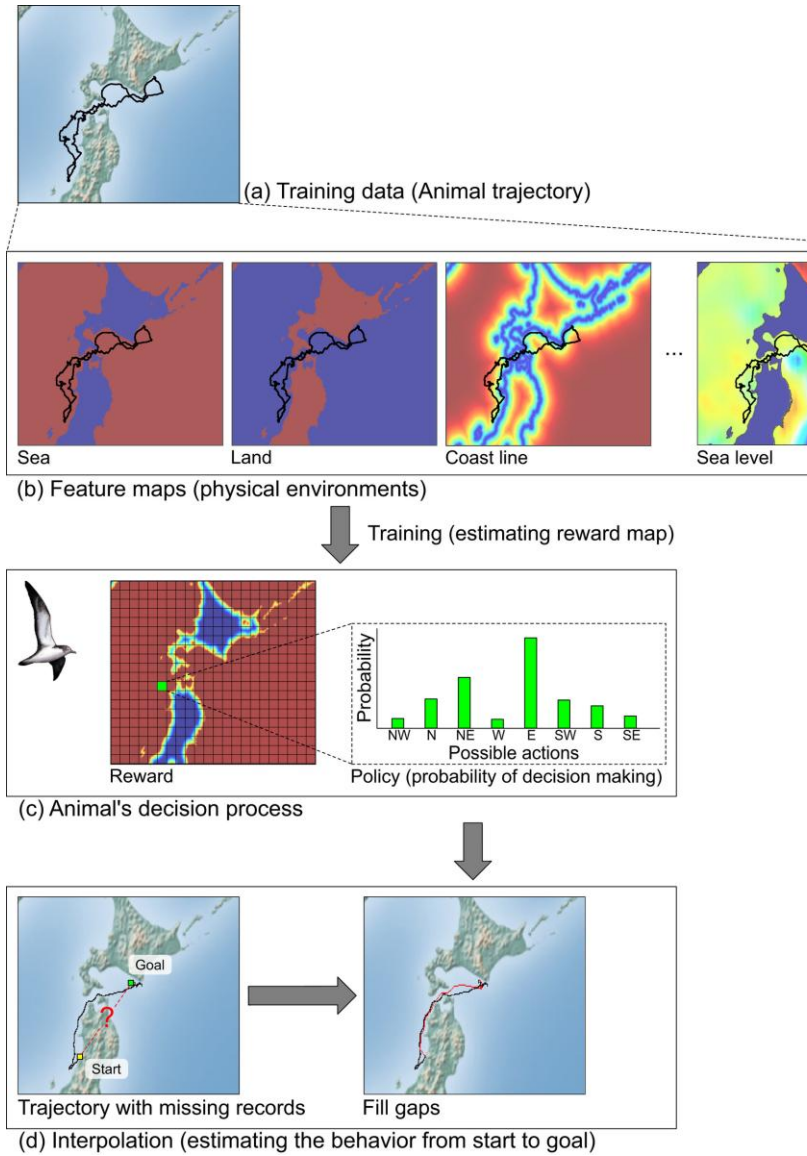
Application: Prediction of the most likely route of shearwater

ミズナギドリ

Background: Scientists track animals with GPS, but the signal often cuts out or batteries save power, leaving **black spaces in the recorded journey**

Objective: The goal was to use inverse RL to **predict the missing parts** of a bird's flight path by learning which environments they prefer to fly over

Contribution: Inverse RL filled black spaces much more accurately than just drawing straight lines and automatically discovered that these birds actively avoid flying over land



Hirakawa et al. (2018). Can AI predict animal movements? Filling gaps in animal trajectories using inverse reinforcement learning. Ecosphere.

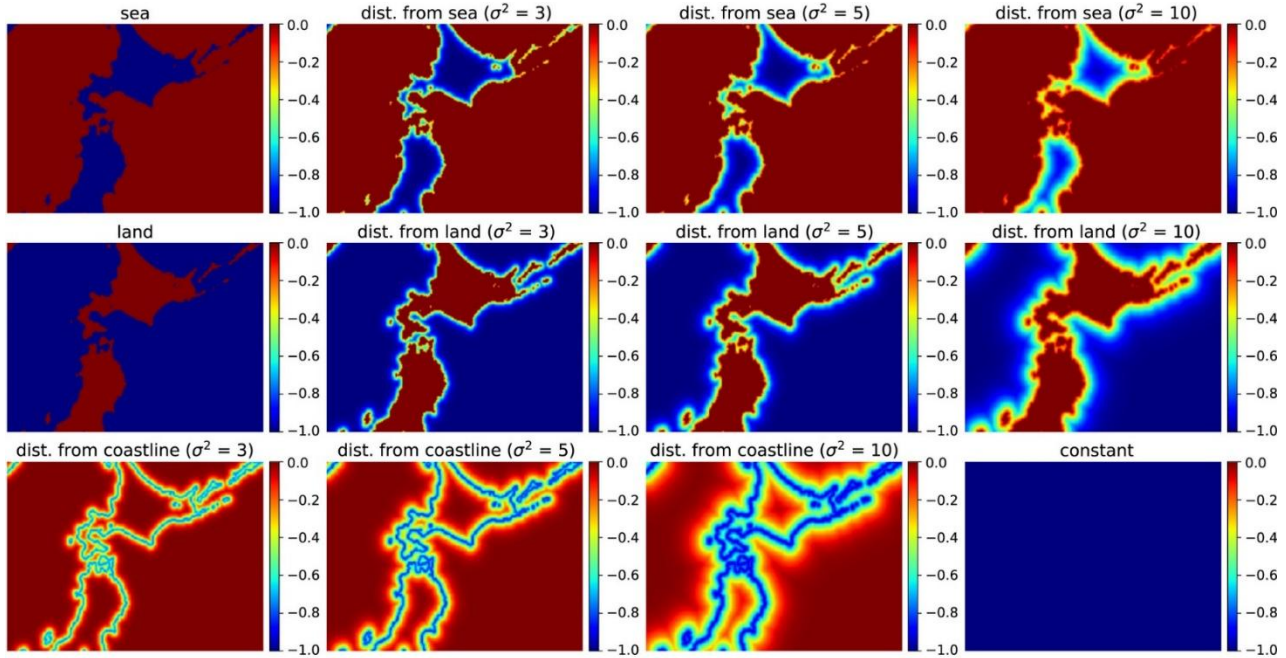
State, action, and reward features

State s is defined as tuple (x, y, z), where x and y are discrete coordinates on a 2D grid (representing physical location) and z is a discrete time step (representing elapsed time)

Action a is defined as the eight possible directions to adjacent grid cells (North, South, East, West, and so on)

Reward features are

- binary maps indicating sea or land
- exponentiated distance map representing proximity to the sea, land, and coastline
- constant map used to model path length preference

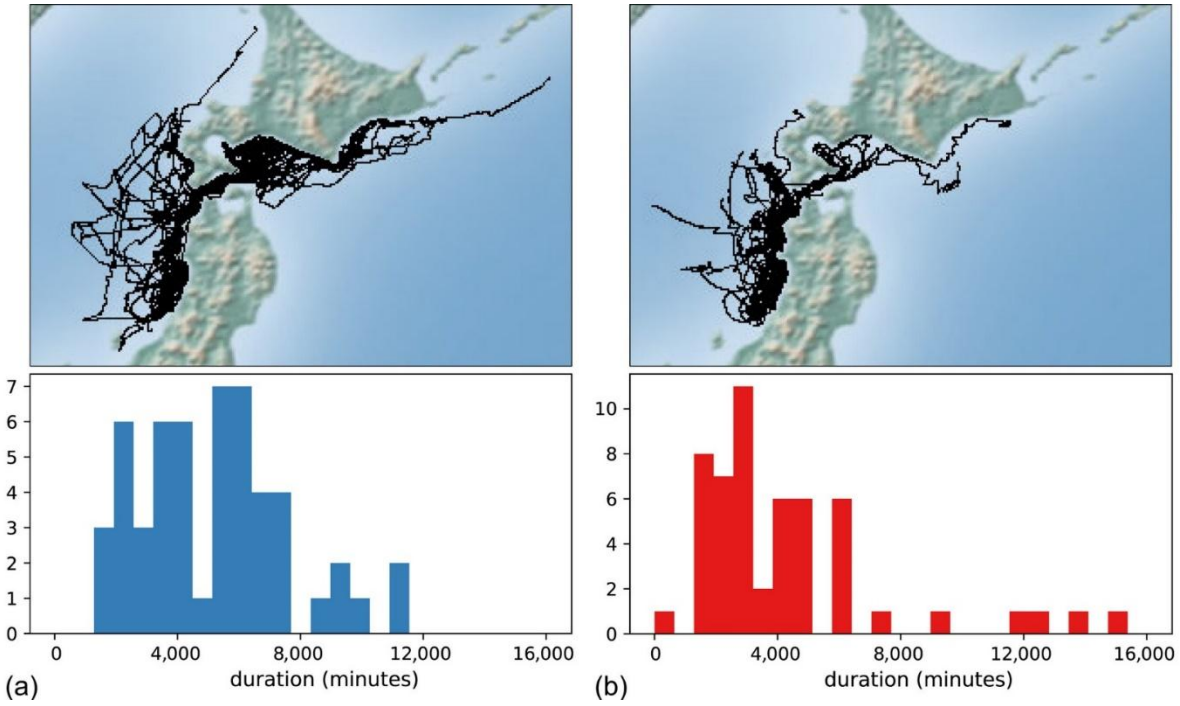


(c)

Data

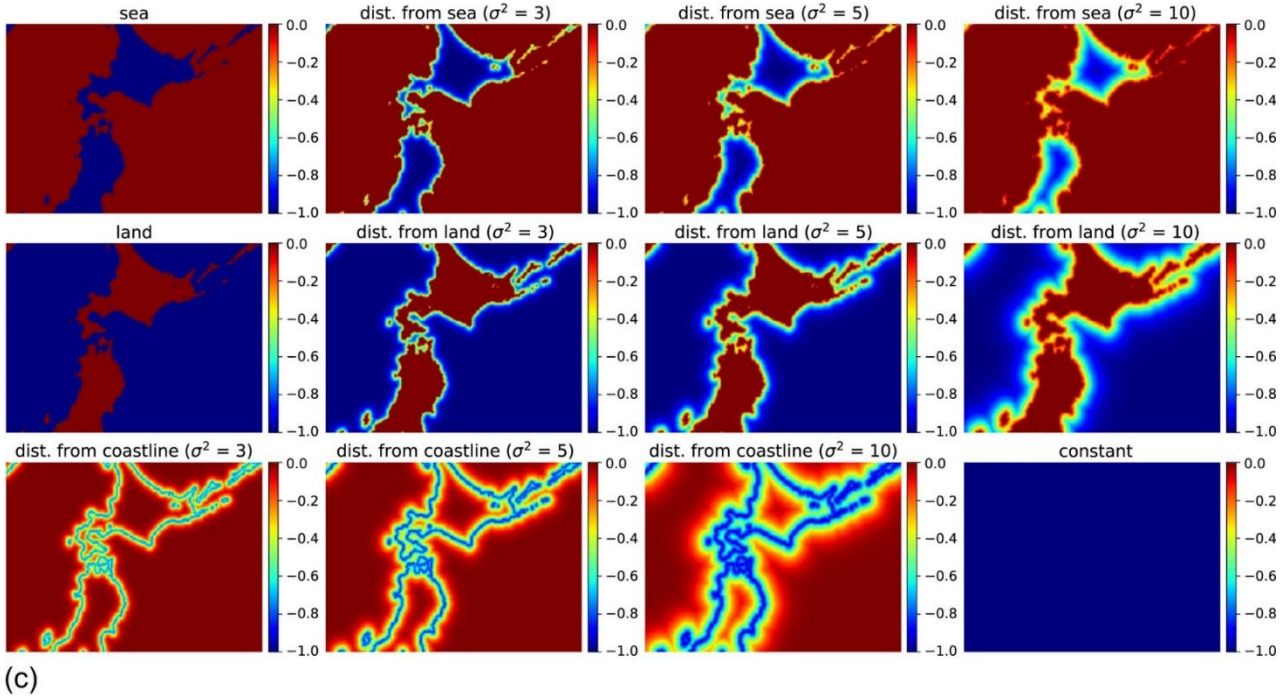
- 106 trajectories (53 males and 53 females)

Male shearwater trajectories and the histogram of trajectory duration



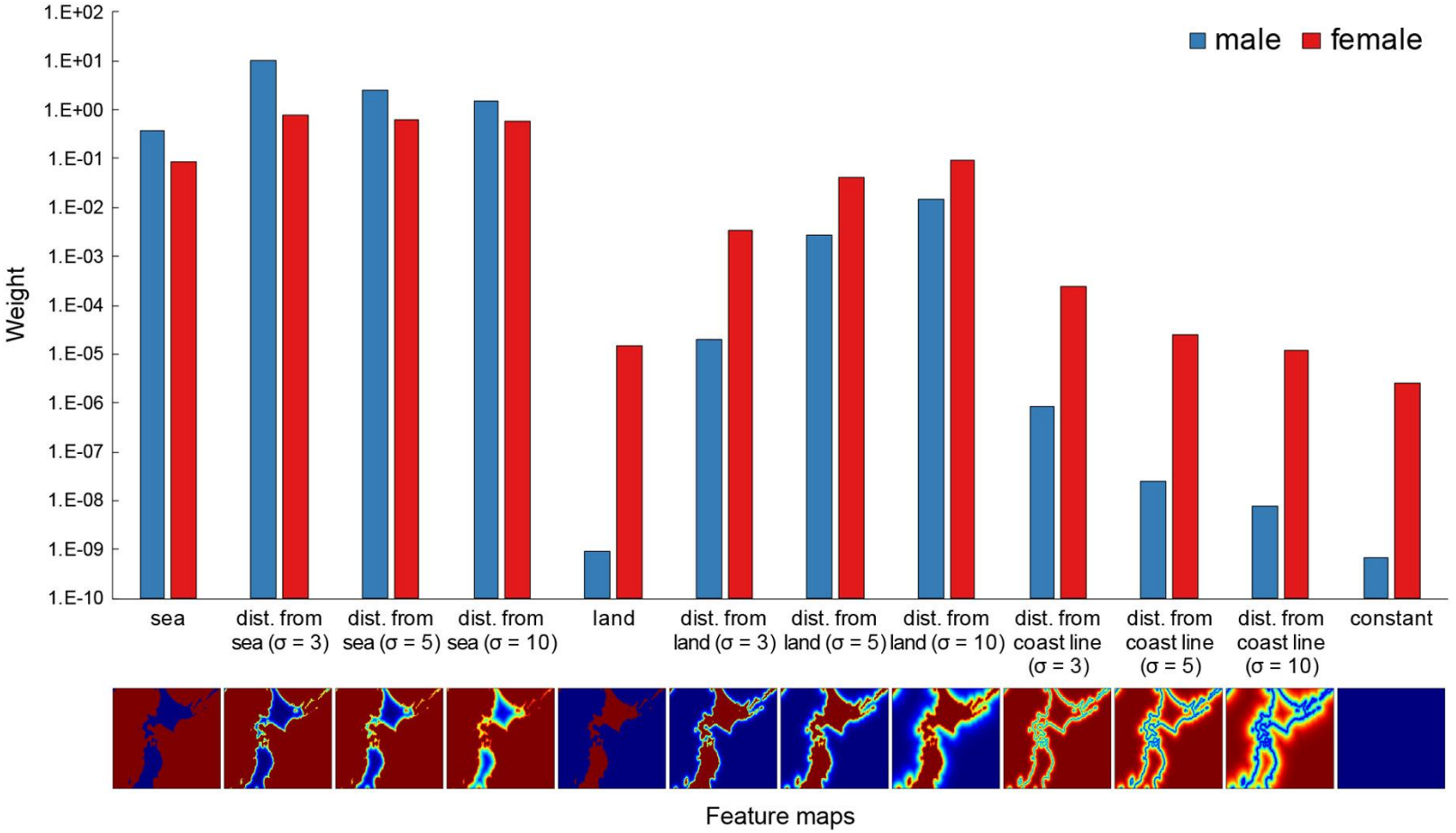
Female shearwater trajectories and the histogram of trajectory duration

reward features



Estimated reward weights

We can see that shearwaters (1) prefer moving over the sea, (2) do not follow the shortest path, and (3) have habitat differences between sexes



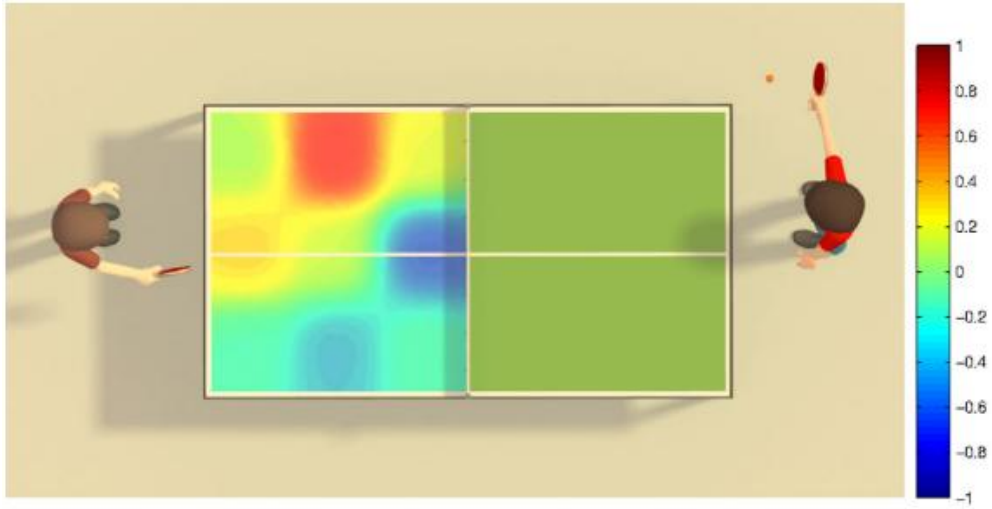
(1) because sea map and distance from sea maps have higher weight

(2) the weight of the constant map is low

(3) Females were more likely to be affected by distance from the coast line

Application: Discovering experts' strategy in sports

Background: Teaching robots to play complex sports like table tennis is difficult because they need to understand strategy (not just how to hit the ball, but where to place it to defeat an opponent)



Objective: Applying inverse RL to analyze real matches and automatically discover the hidden rules or preferences expert players use to decide their next shot

Contribution: Inverse RL successfully identified different playing styles and skill levels by decoding the underlying strategies of human players from their movement



Muelling et al. (2014). Learning strategies in table tennis using inverse reinforcement learning. Biological Cybernetics, 108(5): 603-619.

State variables

State vector

$$\boldsymbol{s} = \begin{pmatrix} \boldsymbol{d}_b \\ |\boldsymbol{v}_b| \\ \theta_{py} \\ \theta_{pz} \\ \boldsymbol{d}_s \\ \boldsymbol{d}_o \\ \boldsymbol{e}_o \\ \boldsymbol{v}_o \\ \boldsymbol{g} \end{pmatrix}$$

… ball's position

… ball's velocity

θ_{py}, θ_{pz}: horizontal and vertical bouncing angles of the ball

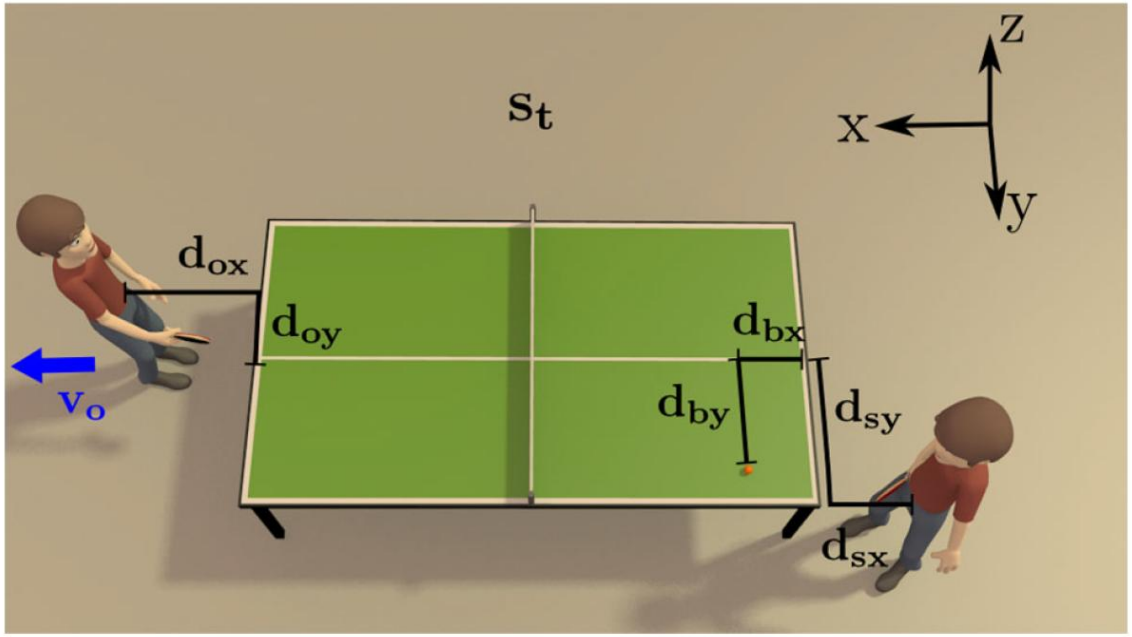
… agent's position

… opponent's position

… elbow position of the opponent

… opponent's velocity

… state of the rally {player serve, opponent serve, not served}

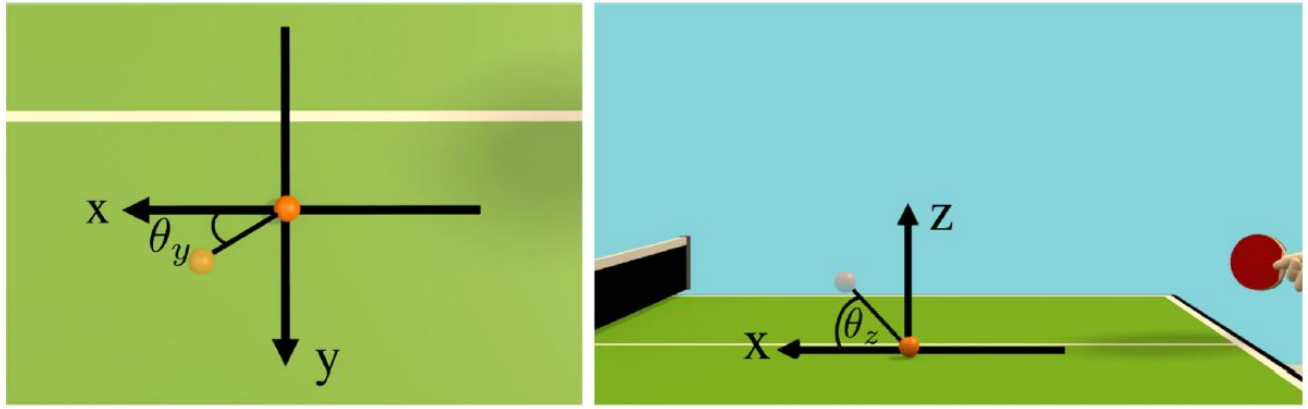


Muelling et al. (2014). Learning strategies in table tennis using inverse reinforcement learning. Biological Cybernetics, 108(5): 603-619.

Reward features $\phi(s, a)$

Well-designed, hand-crafted features are provided by the experimenter

- bouncing angles
- position on the table
- velocity of the ball
- 1 if the ball was a smash, otherwise 0
- distance between the bouncing point and the edge of the table
- distance between the bouncing point and the opponent
- closeness of the ball to the elbow of the opponent
- Movement direction of the opponent
- 1 for winning, -1 for losing, otherwise 0



(a) θ_y (b) θ_z

Fig. 5 The bouncing angles θ_y and θ_z in the xy- and xz-surface define the orientation of the ball. While θ_z corresponds to the horizontal bouncing angle, θ_y corresponds to the direction of the ball and thereby defines if the ball is played cross to the *left*, cross to the *right* or straight

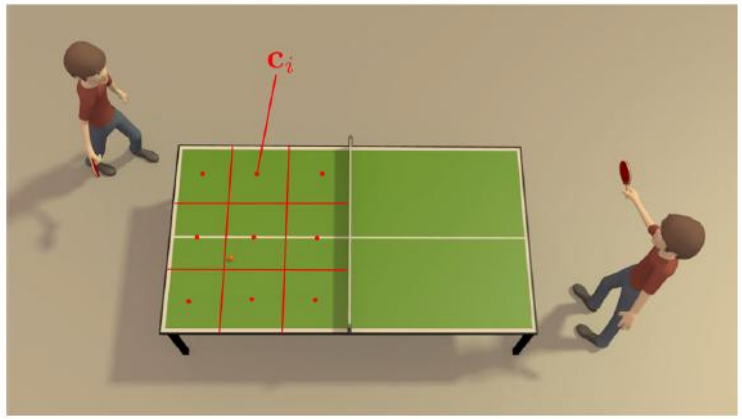
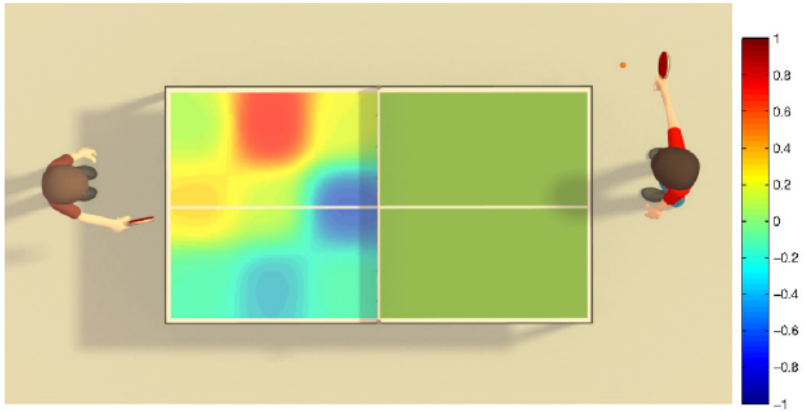


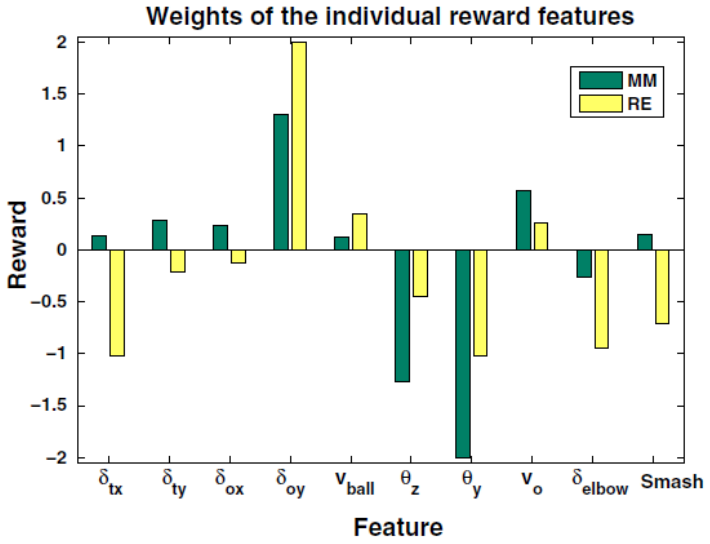
Fig. 4 In order to compute the table preferences on the opponent's court, the table was divided into nine cells. Each cell was assigned a center (*red points*) $\mathbf{c}_i$ (color figure online)

Muelling et al. (2014). Learning strategies in table tennis using inverse reinforcement learning. Biological Cybernetics, 108(5): 603-619.

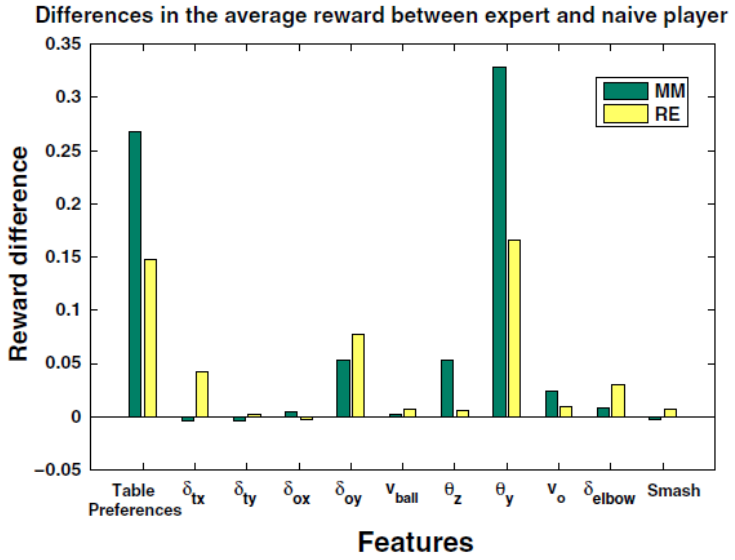
Resulting parameter values for the individual features



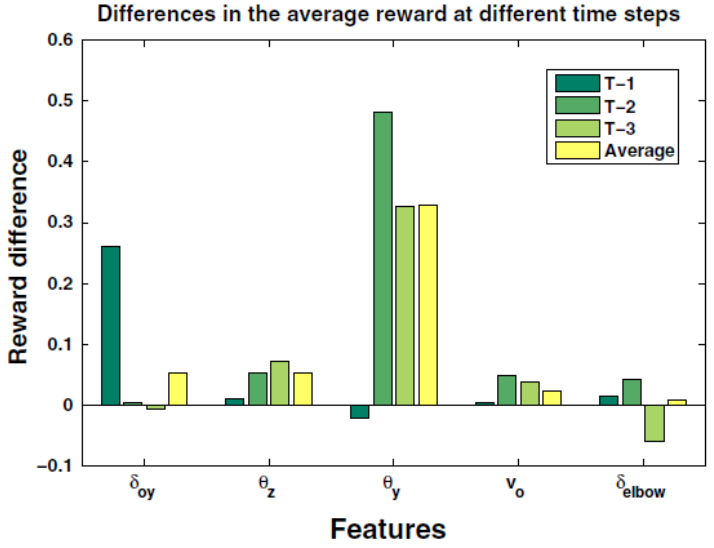
(a) Reward function for table preferences



(b) Reward feature weights



(c) Average reward differences



(d) Reward differences features at different time steps

a. table preference: The heatmap illustrates the learned reward for the ball's bouncing point, showing a clear preference for aiming at the opponent's background area

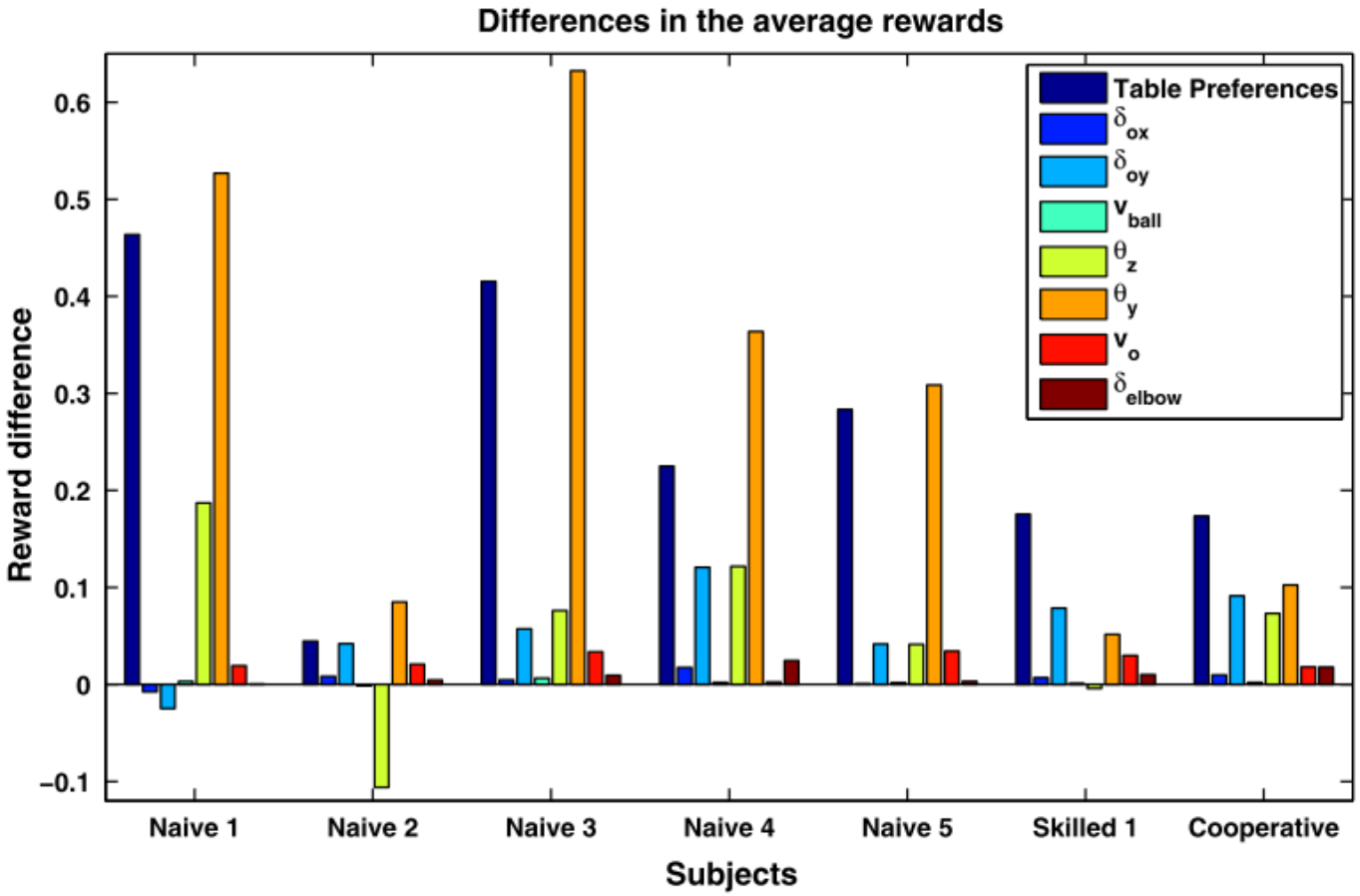
b. reward feature weight: the bars indicate which factors the model identified as most important

c. expert vs. naïve differences: large differences in features like "distance to opponent" δ_{oy} and "bouncing angle" θ_y identify the key strategies used by experts to win

d. temporal difference: the strategic gap becomes most significant immediately before the point is decided

Individual player preferences

The difference in average rewards between the expert and each specific suboptimal player



1. This graph shows which strategic elements (e.g., aiming for the opponent's elbow vs. maximizing distance) contributed most to the skill gap for each specific opponent
2. The player "Naïve 2" had a strategy similar to the expert, indicated by shorter bars, despite being a novice player

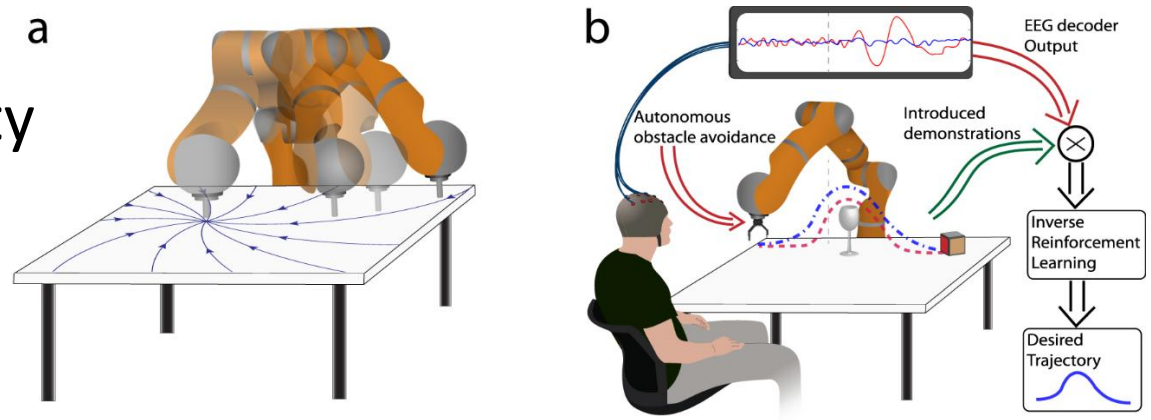
Muelling et al. (2014). Learning strategies in table tennis using inverse reinforcement learning. Biological Cybernetics, 108(5): 603-619.

Application: Brain Computer Interface (BCI)

Background: Controlling robotic arms via BCI is cumbersome and imprecise. While autonomous avoidance exists, it lacks personalization to the user's risk preference

Objective: To customize robot behavior using inverse RL weighted by error-related brain signals (ErrP) detected during observation

Contribution: Developed ErrP-weighted inverse RL that uses brain signals as reliability weights, enabling the robot to rapidly learn personalized and safe trajectories robust to BCI noise



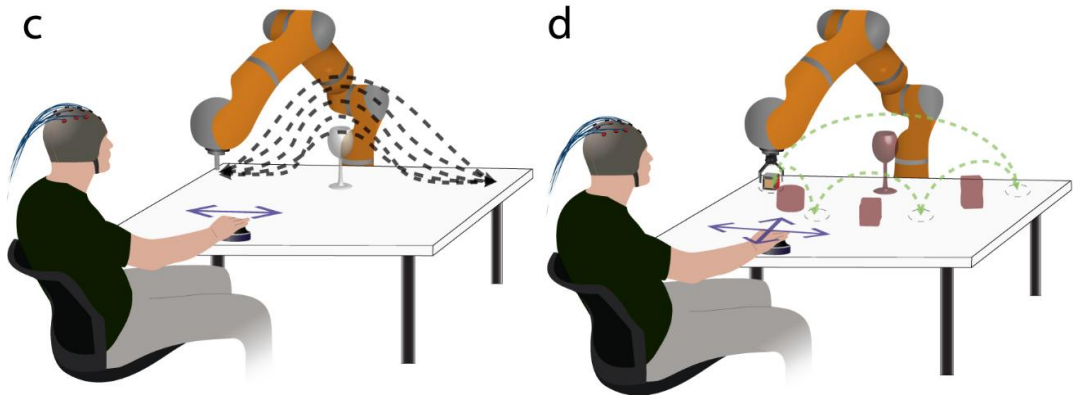
Batzianoulis et al. (2021). Customizing skills for assistive robotic manipulators, an inverse reinforcement learning approach with error-related potentials. Communications Biology 4, 1.

Experiments

The study validated the ErrP-weighted inverse RL approach with 13 subjects through two experiments:

Obstacle avoidance (c): The robot arm moved autonomously side-to-side avoiding a central obstacle. Subjects released a joystick to signal an error whenever they perceived a risk of collision

Pick-and-place (d): Subjects tasked the robot with grasping and moving objects to specific targets on a cluttered table, testing the system's ability to handle more complex, daily-life scenarios

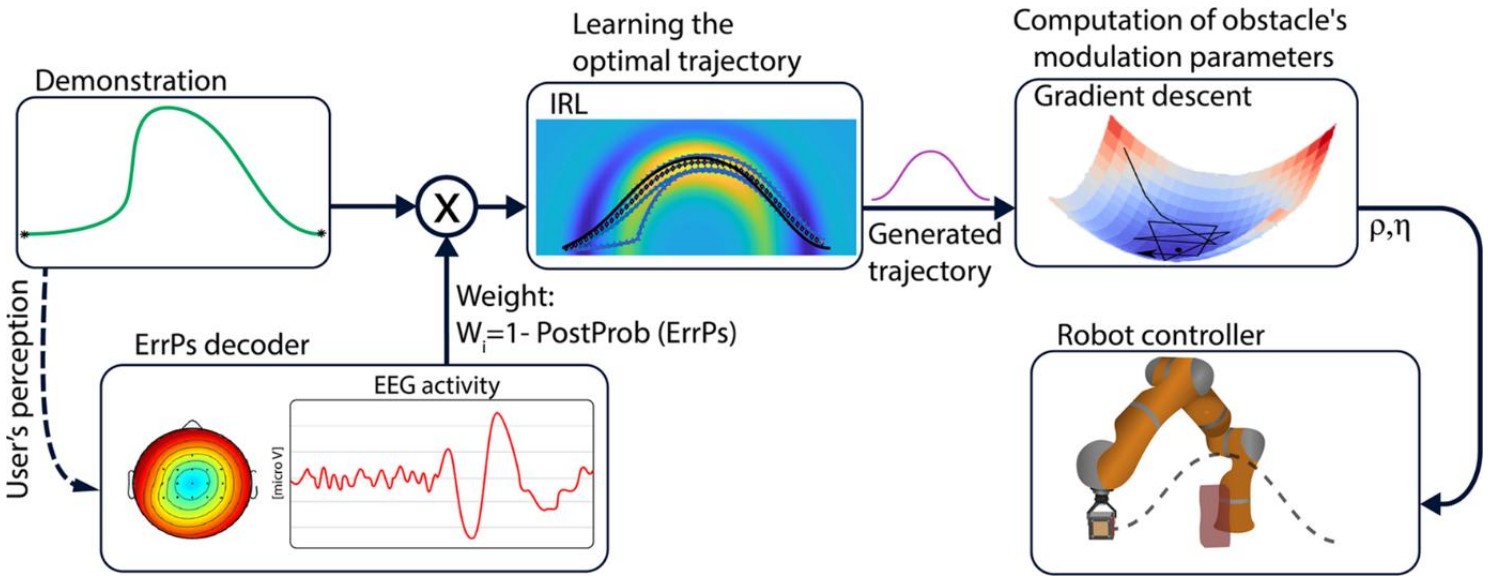


Information flow for training robot controller

For every robot trajectory demo, the system calculates a weight $W = 1 - P_{\mathrm{ErrP}}$ based on the posterior probability of detecting an ErrP in the user's EEG

The weighted demonstration is fed into a Gaussian Process-based inverse RL module, which infers a reward function and generates an optimal trajectory

Finally, the system updates the dynamical system parameters to produce future trajectories that better match the user's safety preferences

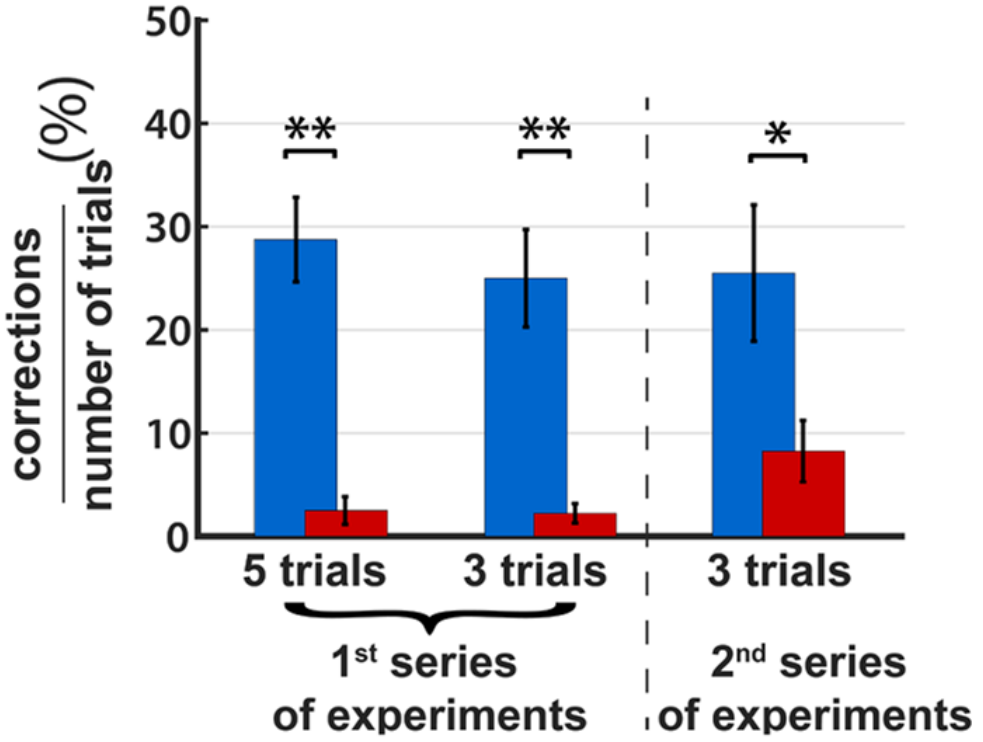


Rapid adaptation to the change of user preferences

The system demonstrated the ability to learn user preferences quickly

The number of corrective actions (joystick releases) required by the user dropped significantly after just 3 to 5 demonstrations

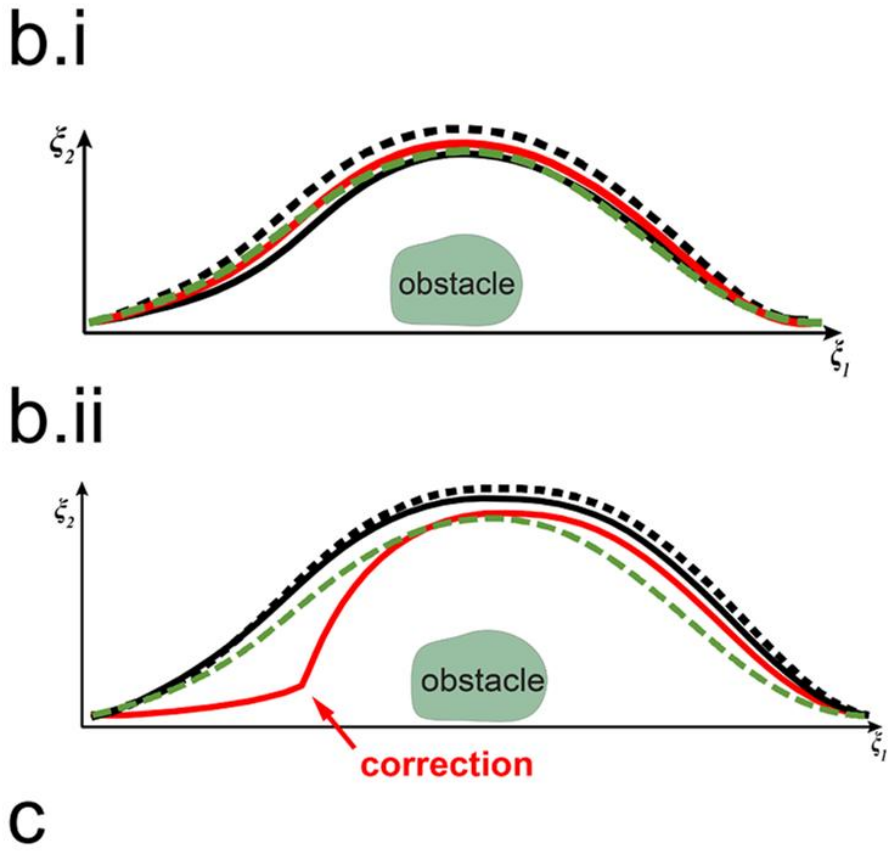
The robot adapted to the desired behavior almost immediately



Robustness to BCI errors

The weighted inverse RL approach showed robustness to the natural instability of EEG signals

Even when the ErrP decoder assigned inconsistent weights, the algorithm successfully filtered this noise and generated smooth, desirable trajectories

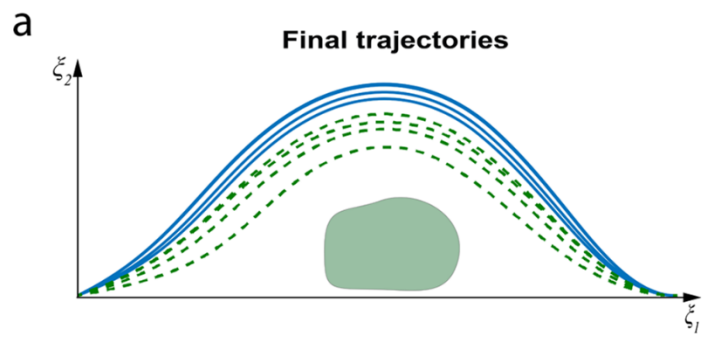


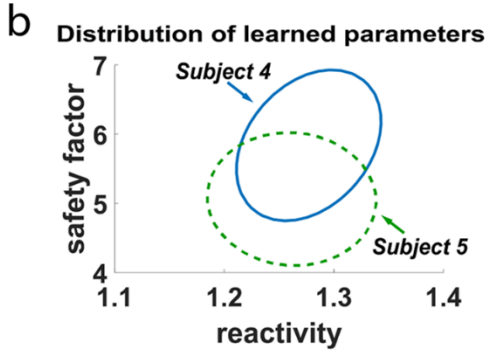
Personalization of skills

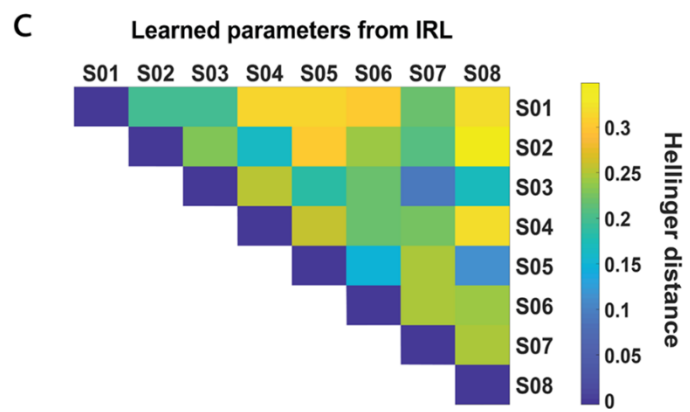
The algorithm successfully encoded distinct individual strategies

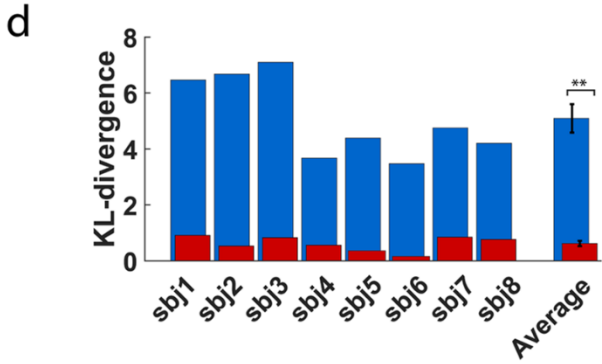
Subject 5 preferred faster, riskier trajectories close to the obstacle, while Subject 4 preferred a wider safety margin

The system learned specific dynamical system parameters for each user to match these styles









Conclusion (1/2): Take-home messages

Inverse RL: infer rewards from behavior when objectives are unobservable

Ill-posedness: multiple rewards can explain the same behavior (reward ambiguity).

MaxEnt Inverse RL: resolves ambiguity by selecting the maximum-entropy distribution consistent with constraints; learning reduces to matching expert feature statistics

Interpretability: learned rewards can serve as an interpretable preference model in healthcare, animal behavior, and sports analytics.

Conclusion (2/2): Practical checklist/limitations

Modeling choices matter: state/action/features define what can be inferred

Data issues: suboptimal demonstrations, partial observability, and sensor noise bias reward estimates

Computation: MaxEnt inverse RL often requires solving a forward RL/planning step repeatedly

Evaluation: validate by trajectory prediction, preference prediction, or downstream policy performance